



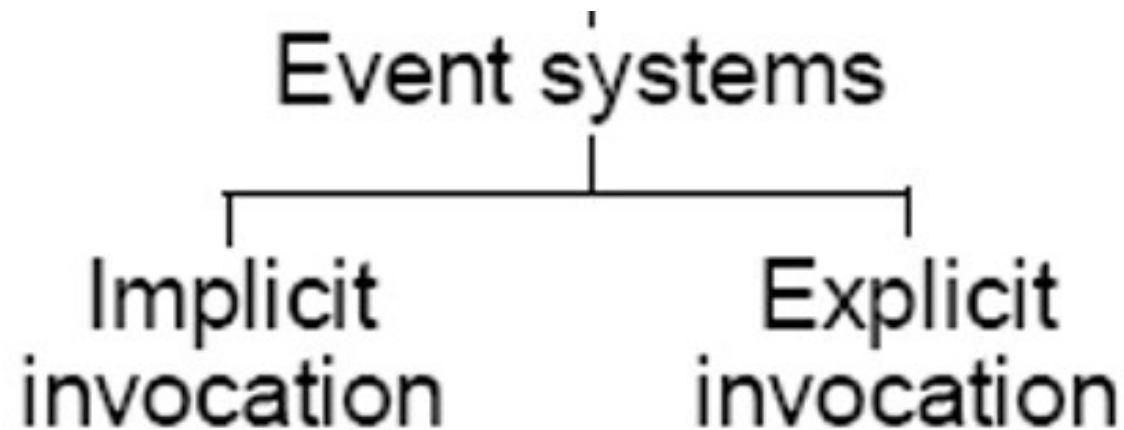
软件体系结构

第6章 事件驱动

赵庆玲

南京理工大学计算机科学与工程学院

Families of Architectural Styles

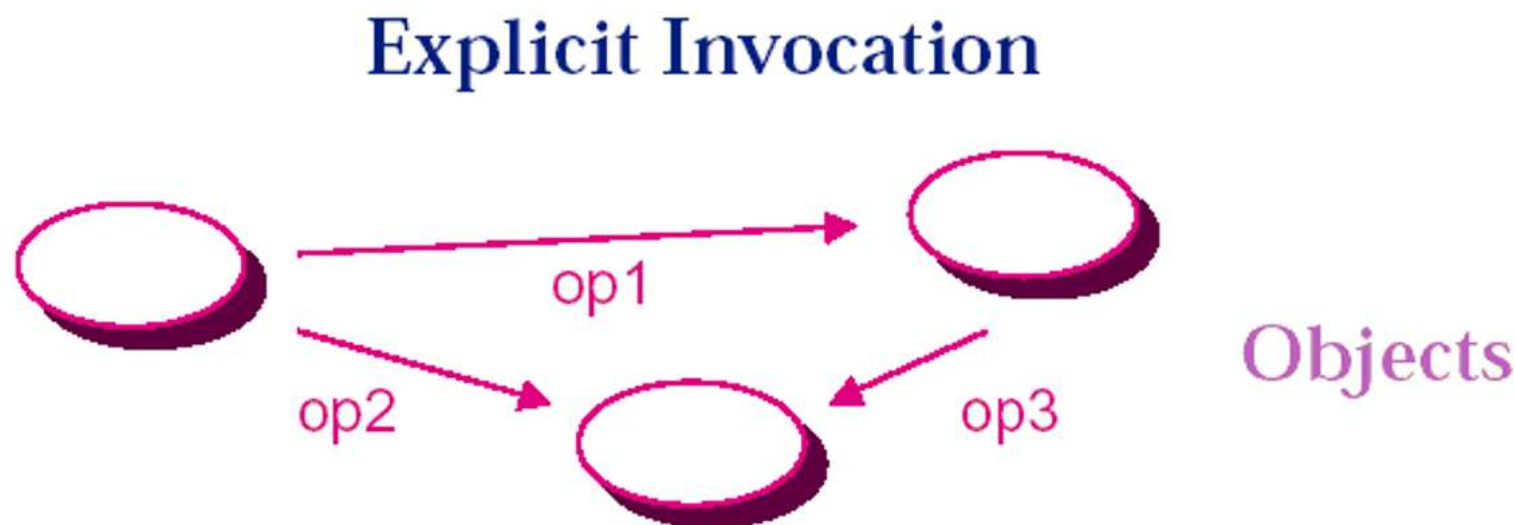


显式调用 Explicit Invocation



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

- 系统中的构件对外提供一个函数集合
 - 如：面向对象中构件相互调用
- 特点：调用者必须知道被调用者的类名、构造方法（包括参数）与要调用的方法（包括参数）

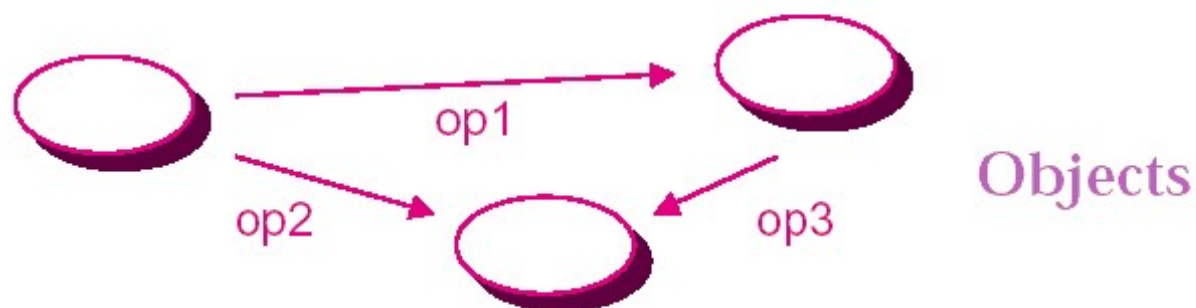


隐式调用vs显式调用

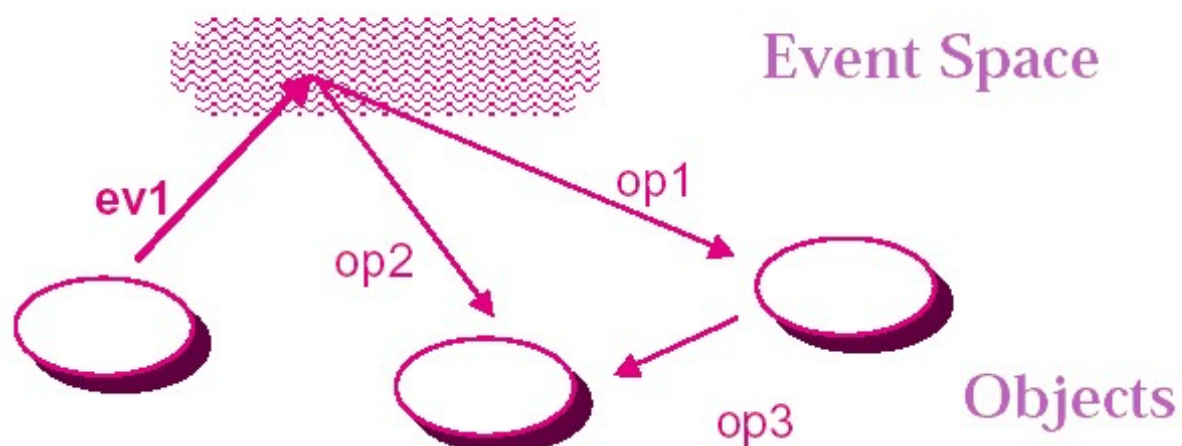


(Implicit versus Explicit Invocation)

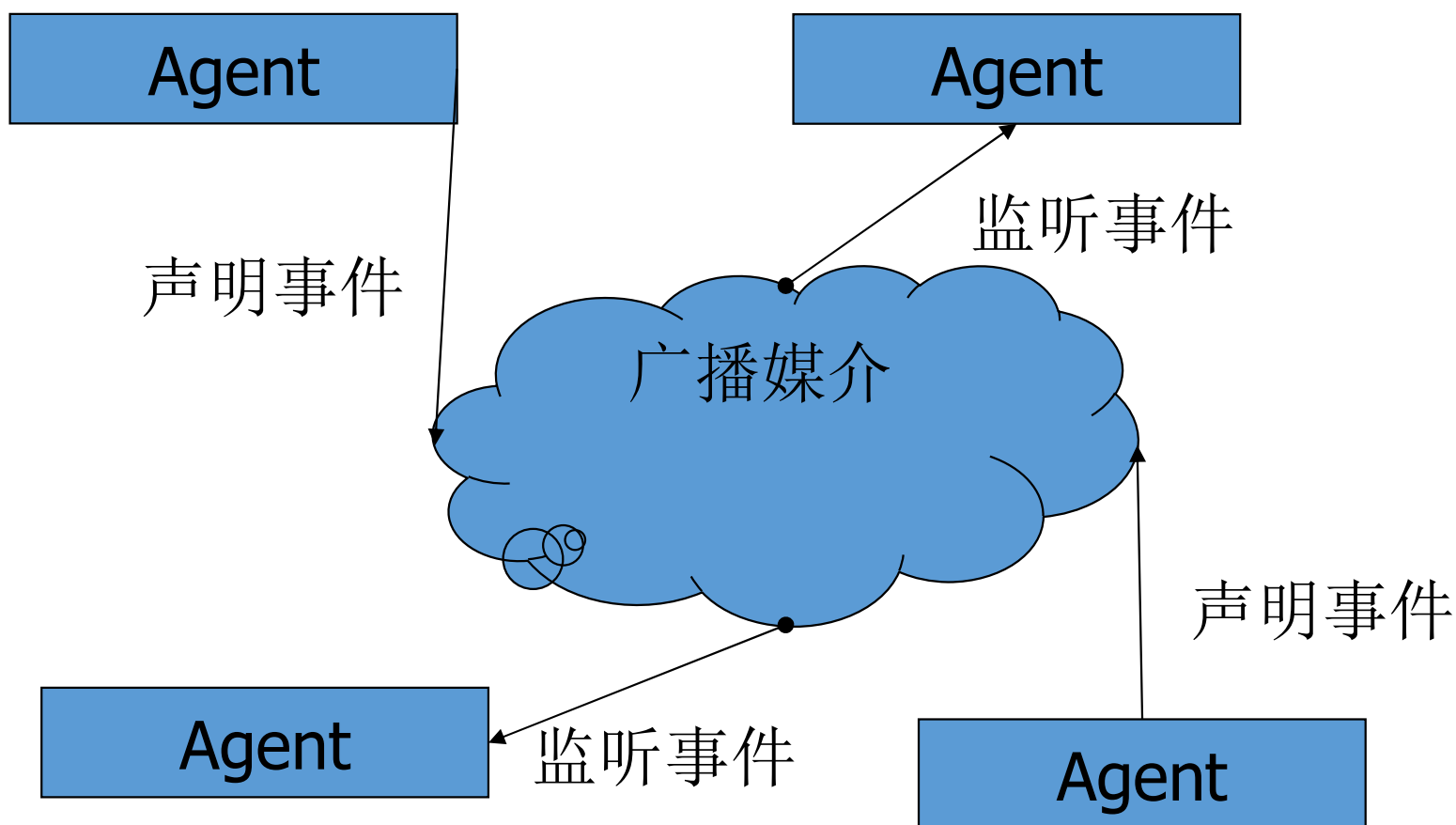
Explicit Invocation



Implicit Invocation



隐式调用示意图



- 事件的发布者并不知道哪些构件会被这些事件影响
 - 不能假定构件的处理顺序，甚至不知道哪些过程会被调用
 - 为此，许多隐式调用的系统也包含显式调用作为构件交互的补充形式。

- 构件不直接调用一个过程，而是触发或广播一个或多个事件。系统中的其它构件中的过程在一个或多个事件中注册，当一个事件被触发，系统自动调用在这个事件中注册的所有过程，这样，一个事件的触发就导致了另一模块中的过程的调用。
- 构件是一些模块，这些模块既可以是一些过程，又可以是一些事件的集合。过程可以用通用的方式调用，也可以在系统事件中注册一些过程，当发生这些事件时，过程被调用。

支持隐式调用的应用系统



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

- 在编程环境中用于集成各种工具
- 在数据库管理系统中确保数据的一致性约束
- 在用户界面系统中管理数据
- 在编辑器中支持语法检查等。

实例：某集成开发环境

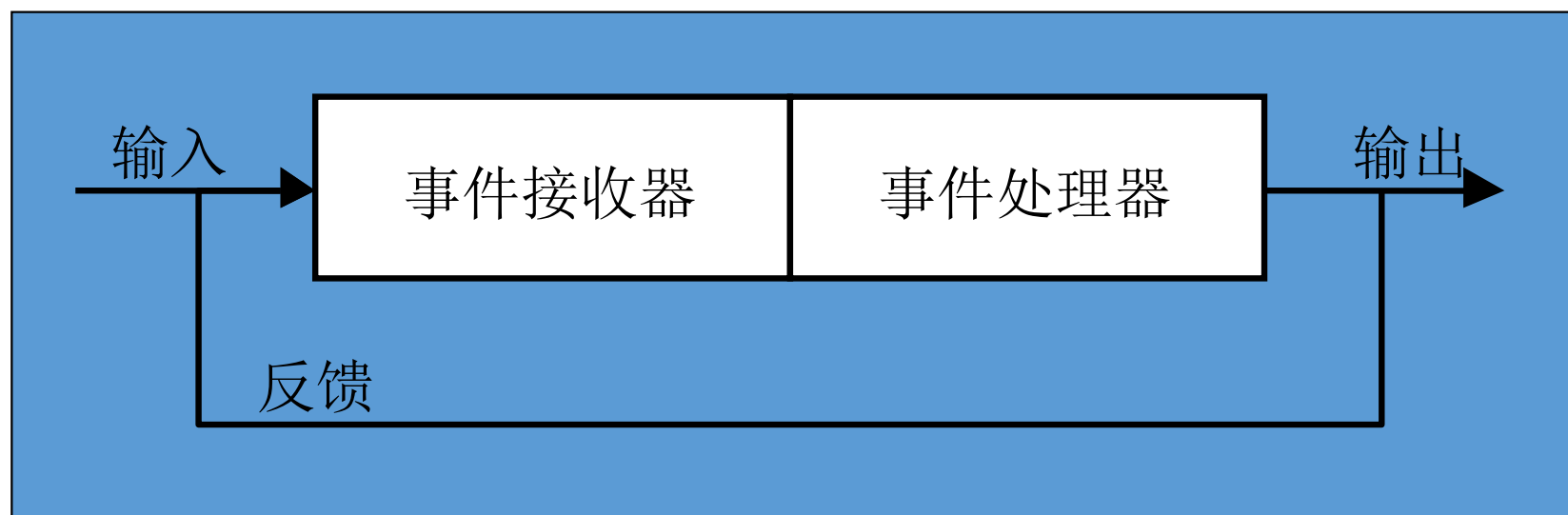


- 在某集成开发环境中，编辑器和变量监视器可以登记相应Debugger的断点事件。
- 当Debugger在断点处停下时，它声明该事件，由系统自动调用处理程序，如编辑程序可以滚屏到断点，变量监视器刷新变量数值。
- Debugger本身只声明事件，并不关心哪些过程会启动，也不关心这些过程做什么处理。

事件驱动风格

事件驱动系统的基本观点是一个系统对外部的表现可以从它对事件的处理表征出来。

我们要认识一个系统，可以给它一个输入事件，然后观察它的输出，从而分析和综合出一个系统。





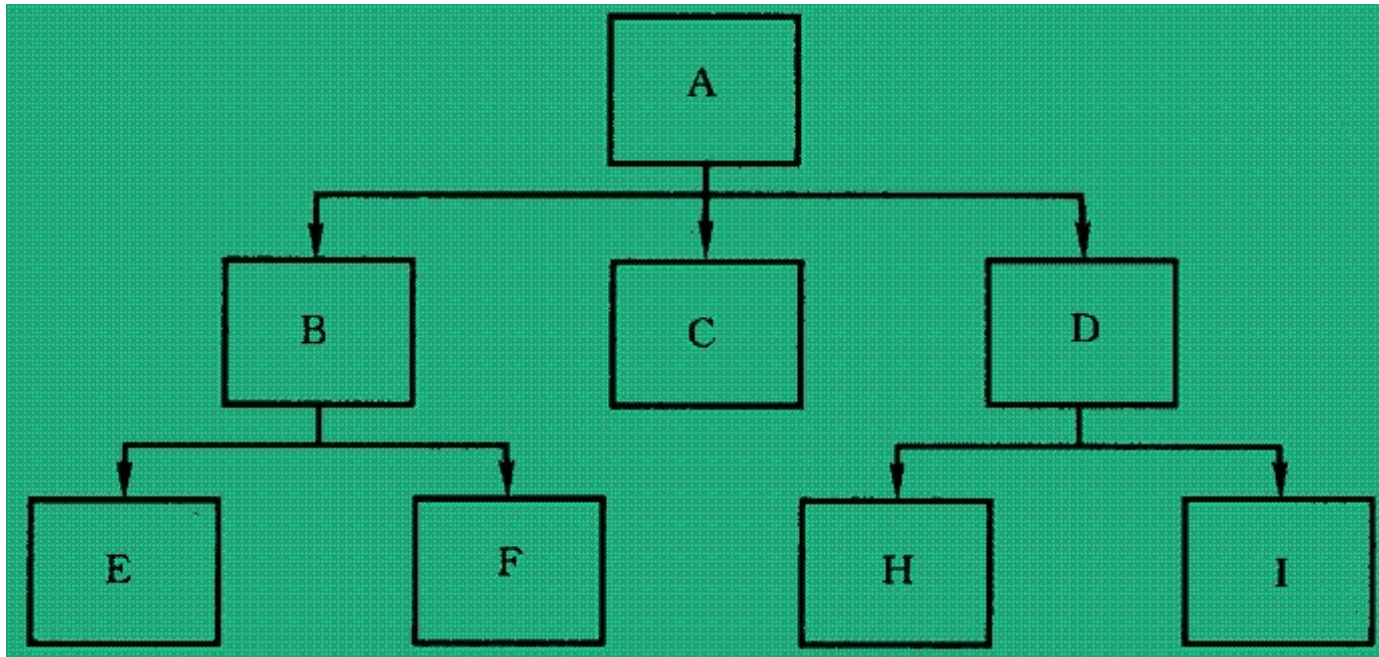
事件驱动系统的特点

- ❑ 系统是由若干子系统或元素所组成的一个整体。
- ❑ 系统有一定的目标，各子系统在某一种消息机制的控制下，为了这个目标而协调行动。
- ❑ 在某一种消息机制的控制下，系统作为一个整体与环境相适应和协调。
- ❑ 在一个系统的若干子系统中，必定有一个子系统起着主导作用，而其它子系统则处于从属地位。
- ❑ 任一系统和系统内的任一元素，都有一个事件收集机制和一个事件处理机制，通过这种机制与周围环境发生作用和联系。

- 1、作为系统中的任何一个子系统，必然有它自己的整体性和相对独立性，它与其他子系统的联系是通过某种消息机制完成的。
- 2、从这种观点来看，过去的结构化、模块化系统所划分的模块不能称为子系统。
- 3、面向对象的事件驱动系统设计方法把软件系统作为一个系统，将其分为一个个子系统来实现。



一个基于事件驱动的软件系统的示意图



- 从系统论的角度来看待描述的对象，合理分解子系统，保证各个子系统的独立性和社会性；
- 无论系统多么复杂，子系统性质的差异多么大，任何子系统都可以按照有无子系统这一性质分为2类：管理系统和执行系统。
- 为了达到系统的目标，系统内的各个子系统通过传递消息和执行消息来协同操作。

- 在一个完整系统中，必须有这样一个子系统，它没有上级，必须收集系统外的事件及下级发出的事件。
- 管理类型的子系统一般不执行具体操作，其主要功能是按照自己的职能指挥下级完成任务，功能性操作一般由执行类型的子系统完成。
- 一般情况下，除最高级管理子系统外，子系统一般是“有问才答”，即使在必要的情况下需要积极寻找事件时，也必须征得上级系统得许可，保证了系统的控制流不会分散。

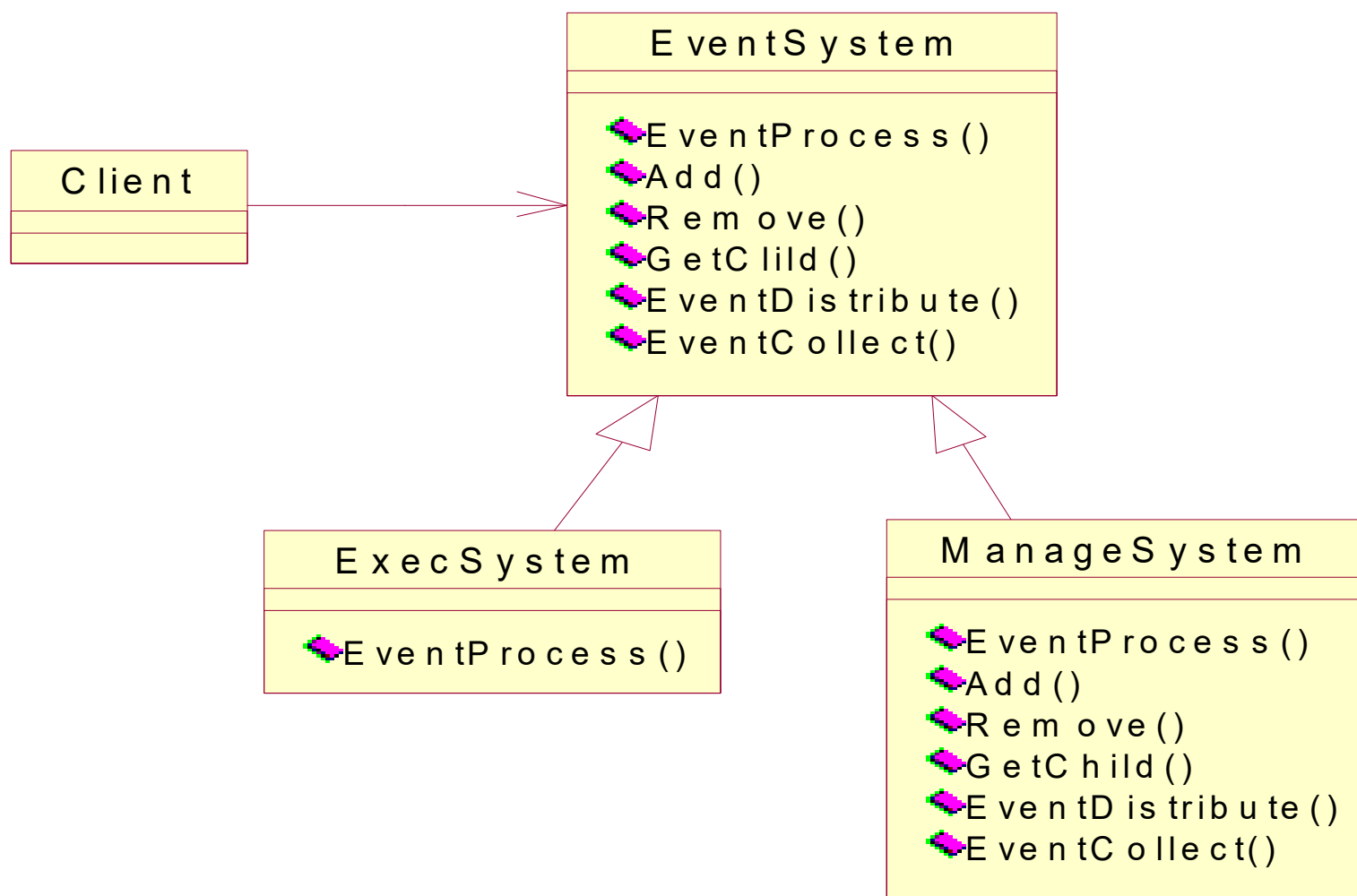


事件驱动模式的基本结构

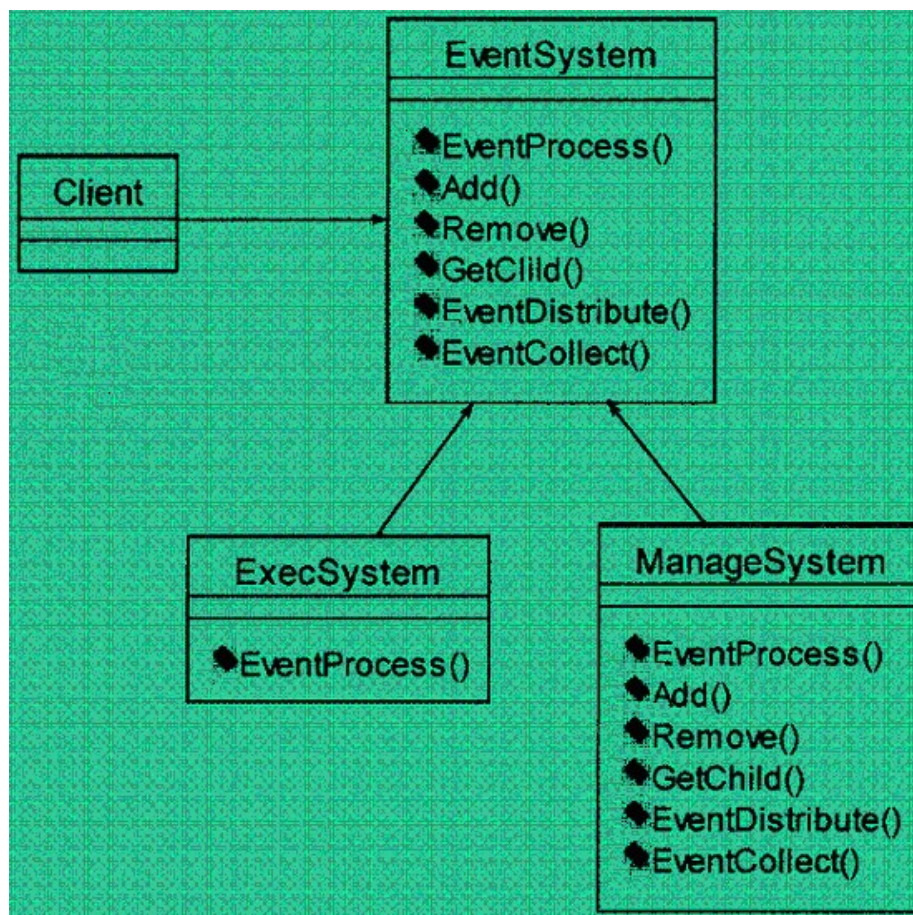
- 事件驱动系统具有某种意义上的递归性，形成了“部分—整体”的层次结构，可以用属性结构加以表示。
- 一个简单的表示方法是为执行系统定义一些类，另外定义一些类作为这些执行系统的容器类，也就是管理系统。

- * 这种方法存在一个问题。使用这些类的代码必须区别对待执行系统与管理系统，而实际情况下对用户而言，它们在很大程度上是一样的。对这两种子系统区别使用，使得整个系统的实现和使用更加复杂。为了简化事件驱动模式系统的设计、实现和使用，可以借鉴设计模式中的结构型 (Composite) 模式，来定义、组织和管理事件驱动模式系统中的执行系统和管理系统。简化的关键就是定义一个接口(或抽象类)，它既可以代表执行系统，又可以代表执行系统的容器(管理系统)。在事件驱动模式系统中，我们定义这个接口的名字为 EventSystem，它声明了一些与事件驱动相关的操作，例如 EventProcess(事件处理器)。同时它也声明了所有的管理系统共享的一些操作，例如 EventDistribute(事件分发器)和 EventCollect(事件收集器)。其结构图可以用UML类图描述如图：

事件驱动模式的基本结构



事件驱动系统的基本结构



事件驱动模式系统的基本结构

1、EventSystem

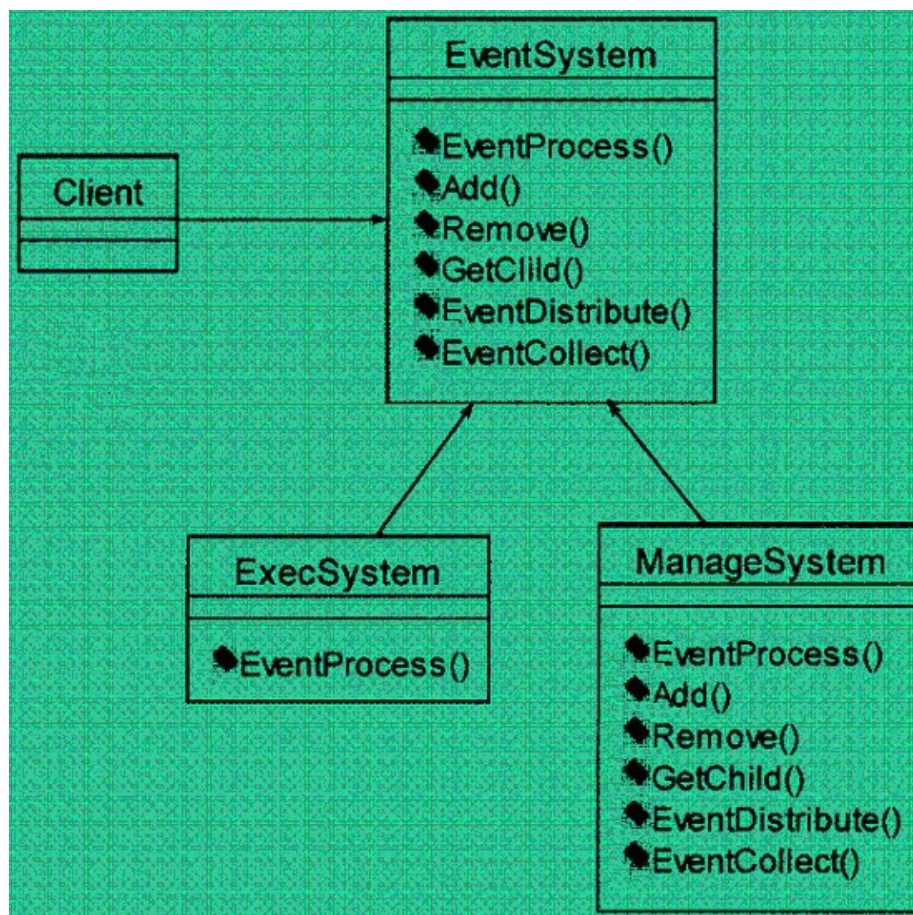
o为事件驱动模式系统中的子系统声明接口；

o在适当时清况下，实现所有子系统共有的接口的缺省行为；

o声明一个接口用于访问和管理事件驱动模式系统的子系统；

o在递归结构中定义一个接口，用于访问一个父系统，并在合适的情况下实现它。

事件驱动系统的基本结构

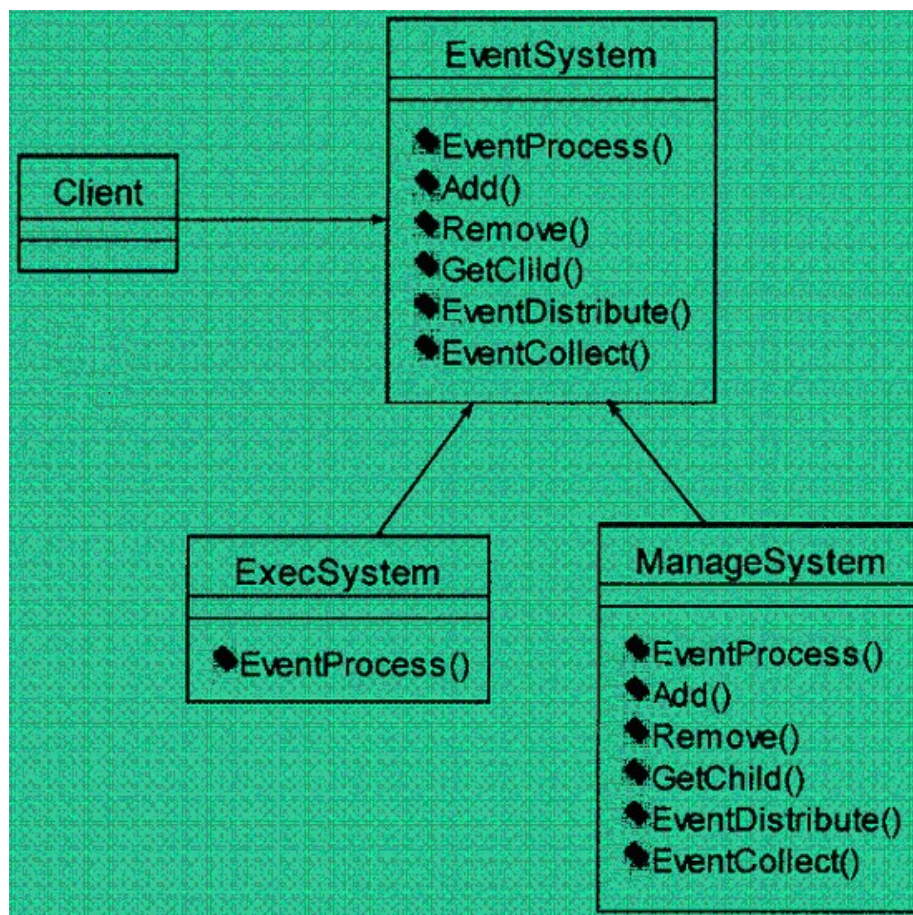


2、ExecSystem

- o在事件驱动模式系统中表示执行子系统，执行子系统是无子系统的系统；
- o定义执行子系统具体的行为。

事件驱动模式系统的基本结构

事件驱动系统的基本结构

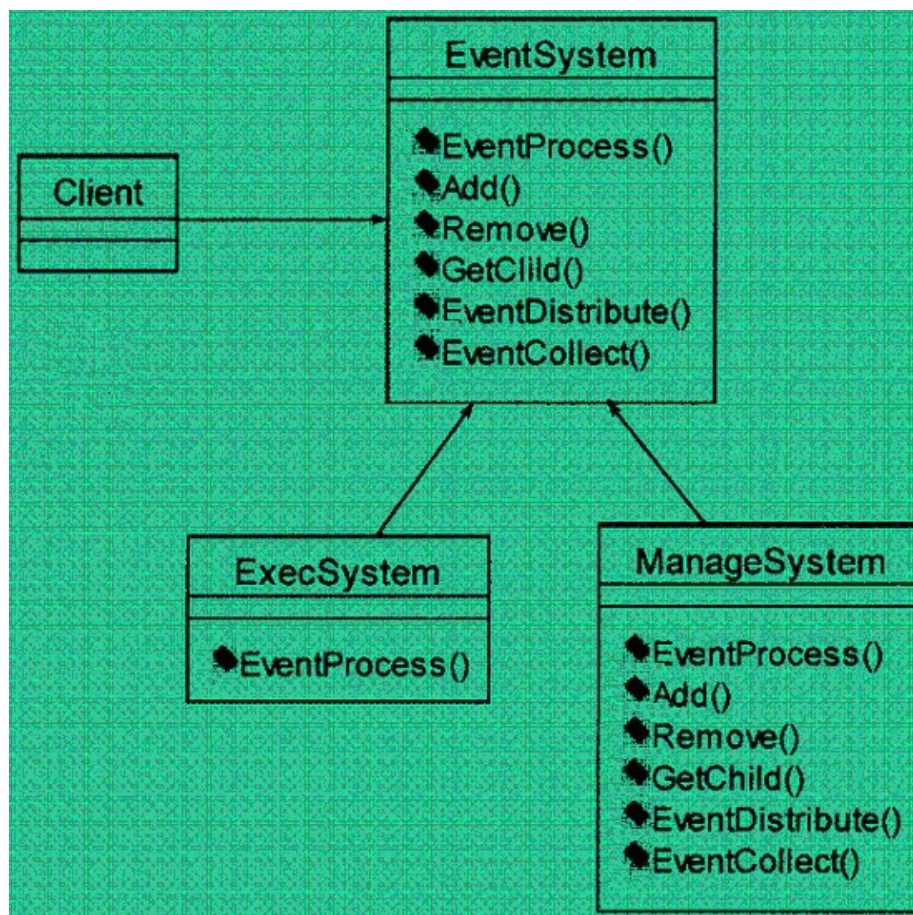


事件驱动模式系统的基本结构

3、ManageSystem

- o 定义管理子系统的基本行为，如事件分发机制和事件收集机制等；
- o 存储其子系统，也就是由它管理的执行子系统或管理子系统；
- o 在 **EventSystem** 接口中实现与子系统有关的操作。

事件驱动系统的基本结构



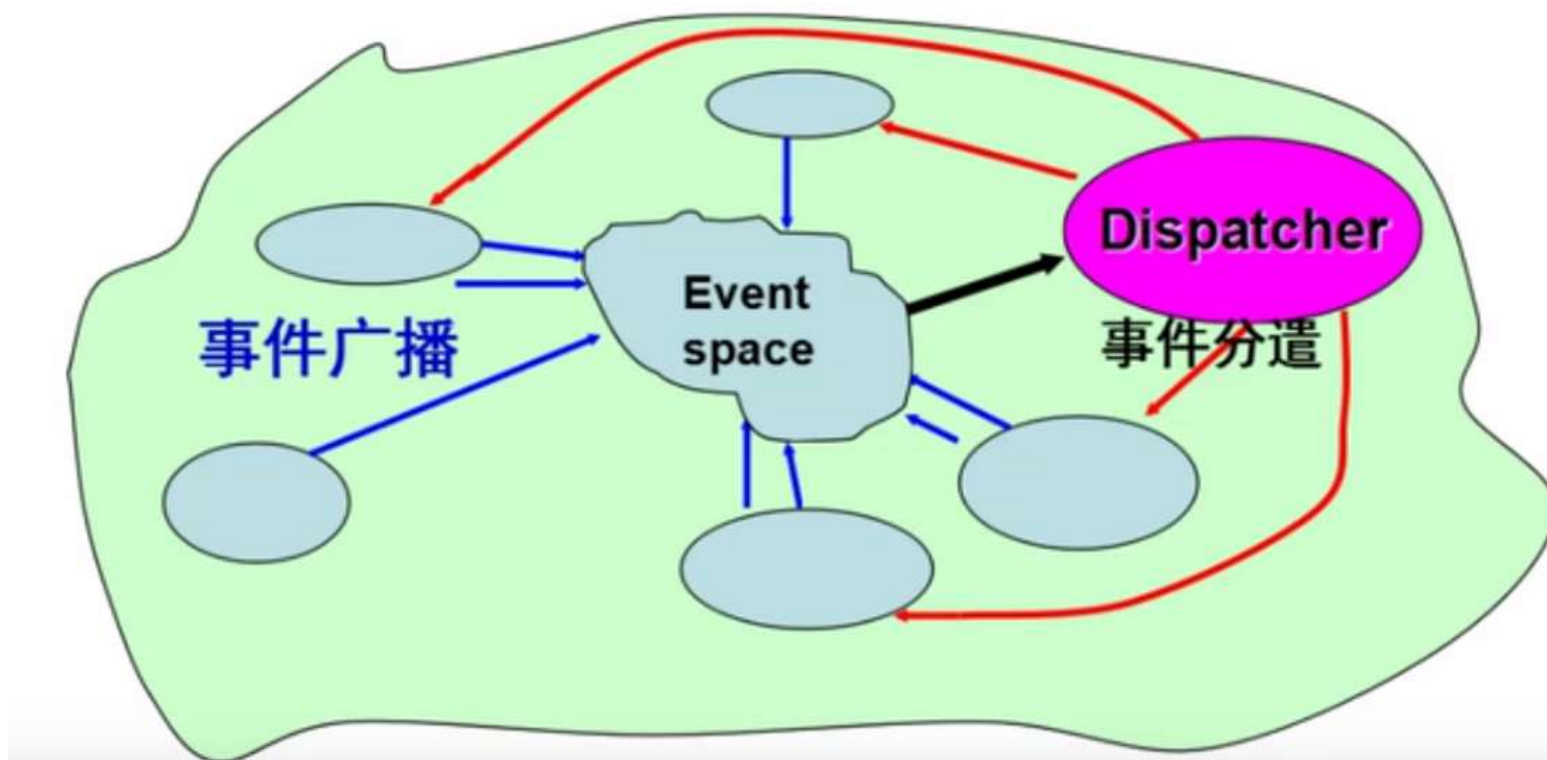
事件驱动模式系统的基本结构

4、Client

通过**EventSystem**操作整个事件驱动模式系统，与系统中的子系统进行交互。如果消息接收者是一个执行子系统，则直接处理请求；如果接收者是管理子系统，它通常将请求以消息的形式发送给它的子系统，在转发请求消息之间或之后，可能执行一些辅助操作。

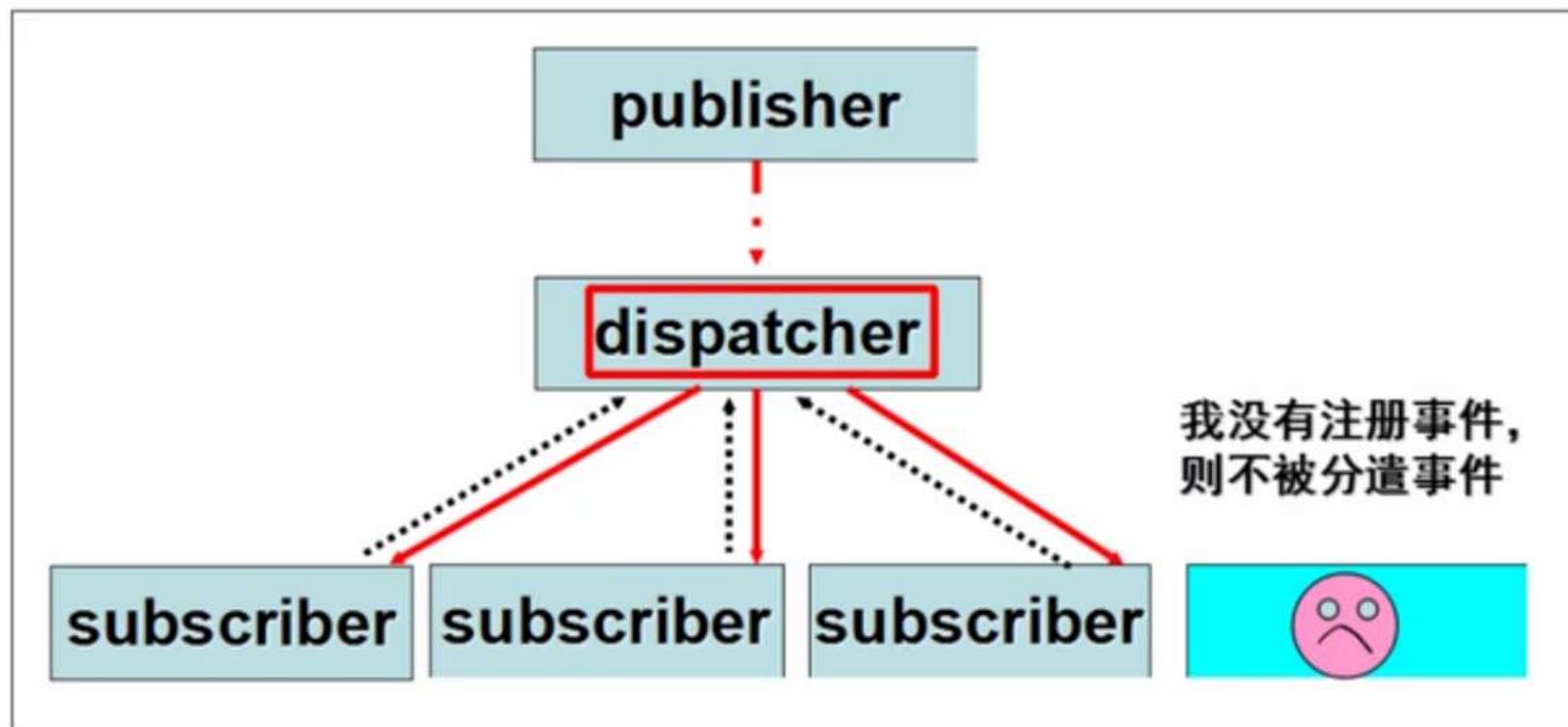
- 如何将事件分发到相关的事件处理程序？两种处理策略：
 - 有独立事件分遣模块
 - 无独立事件分遣模块
- 独立事件分遣模块Dispatcher
 - 接收所有事件，将事件分遣到相关模块

有Dispatcher



- 两种分遣方式:
 - 广播
 - 发布/订阅策略

发布/订阅策略

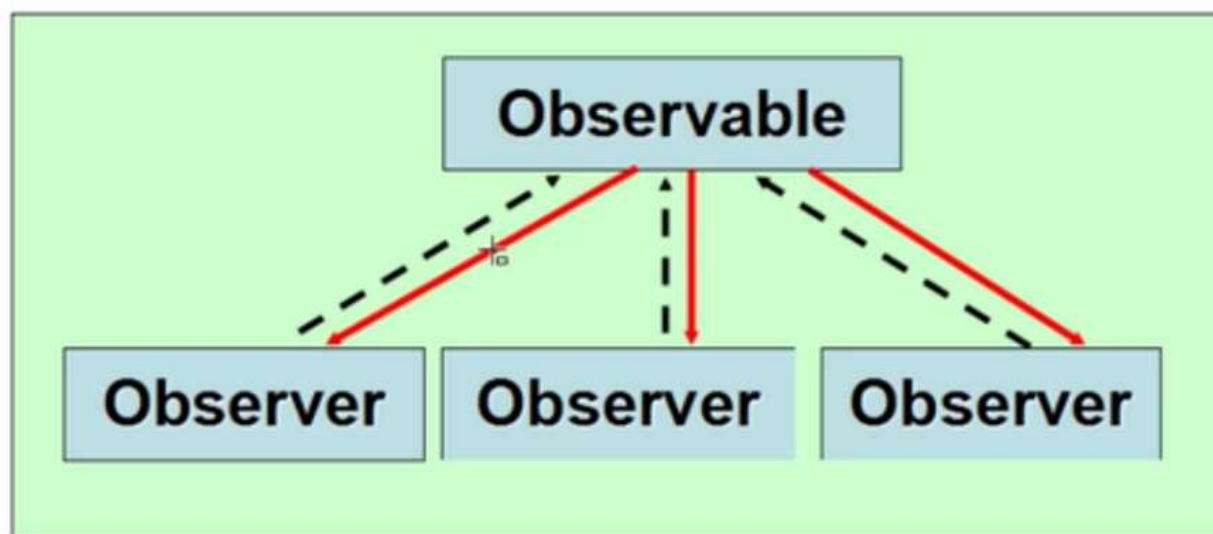


发布事件
订阅事件
分遣事件

无Dispatcher

- Observable/Observer模型
 - 每个模块允许其他模块对发送的事件感兴趣
 - 只讲事件发送给注册者
- 观察者模式

Observable/Observe模型



图例:

.....➡ 注册事件

————➡ 发送事件



事件驱动风格的优点

- 适合于描述系统族
 - 在属于同一族的任何系统中，系统的高级管理子系统的描述是完全类似的，便于重用；
- 容易实现并发处理和多任务操作
 - 由于最高管理子系统牢牢的掌握着控制权，又因为各同级子系统一般不直接发生关系
- 具有良好的可扩展性——重用、改进
 - 设计者只需为某个对象注册一个事件处理接口就可以将该对象引入整个系统，同时并不影响其它的系统对象。



事件驱动风格的优点

- 类结构简明
 - 定义了包含执行子系统和管理子系统的类层次结构。
- 简化客户代码
 - 使整个系统的设计更具有一般化。

- 构件放弃了对系统计算的控制
 - 一个构件触发一个事件时，不能确定其它构件是否会响应它。即使它知道事件注册了哪些构件的构成，它也不能保证这些过程被调用的顺序。
- 数据交换的问题
 - 有时数据可被一个事件传递，但另一些情况下，基于事件的系统必须依靠一个共享的仓库进行交互。这也使全局性能和资源管理便成了问题。
- 正确性的推理存在问题
 - 这是由于过程的语义必须依赖于被触发事件的上下文约束。



事件驱动风格与面向对象

- 基于面向对象模式的系统由多个封装起来的对象构成，对象之间通过消息传递实现通信，而事件驱动正是对消息传递机制的一种实现。
- 基于事件驱动模式的系统往往都是面向对象的。

事件驱动风格实例

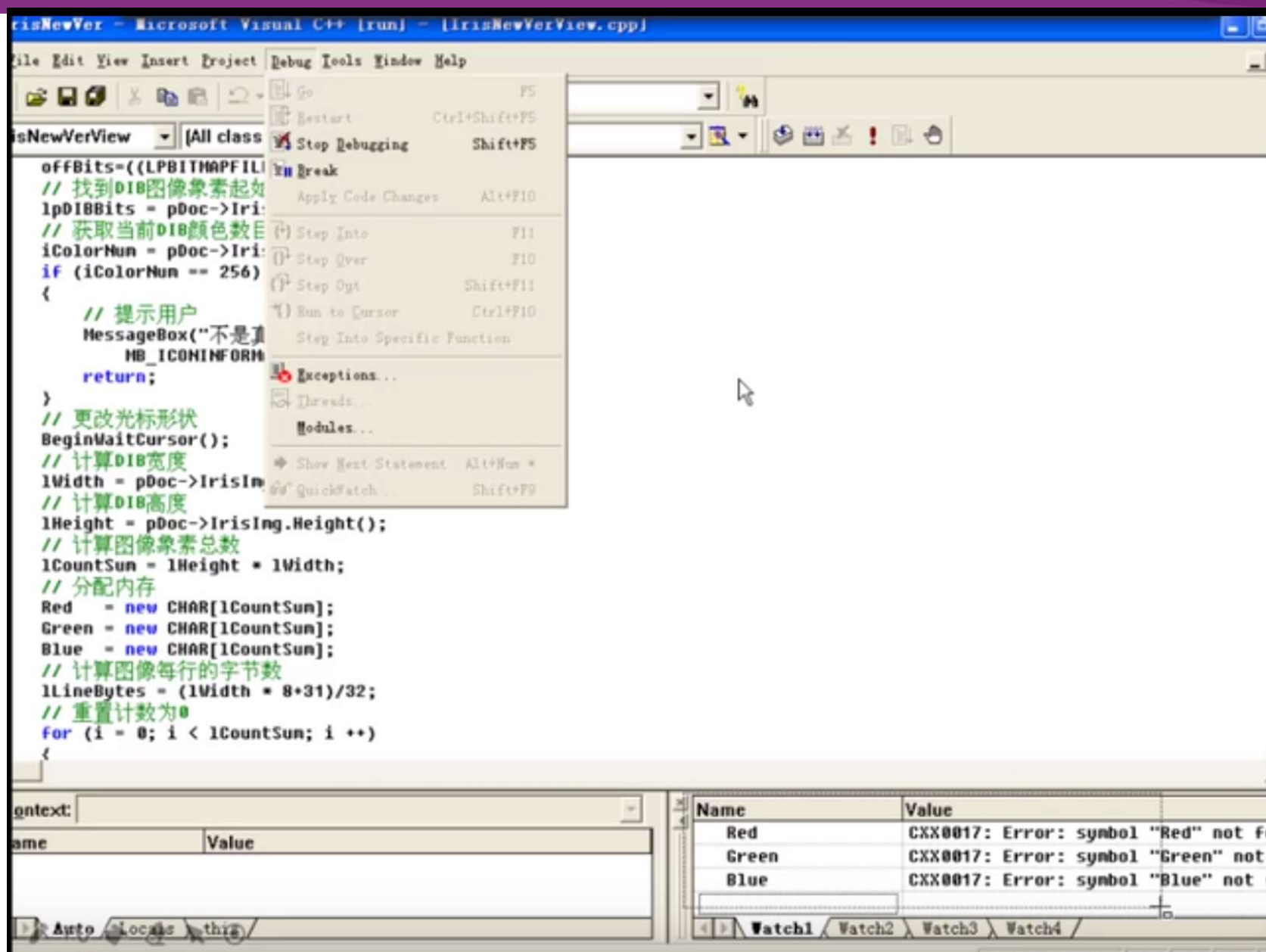


Example: 考虑C, C++ 或 Java的集成开发环境

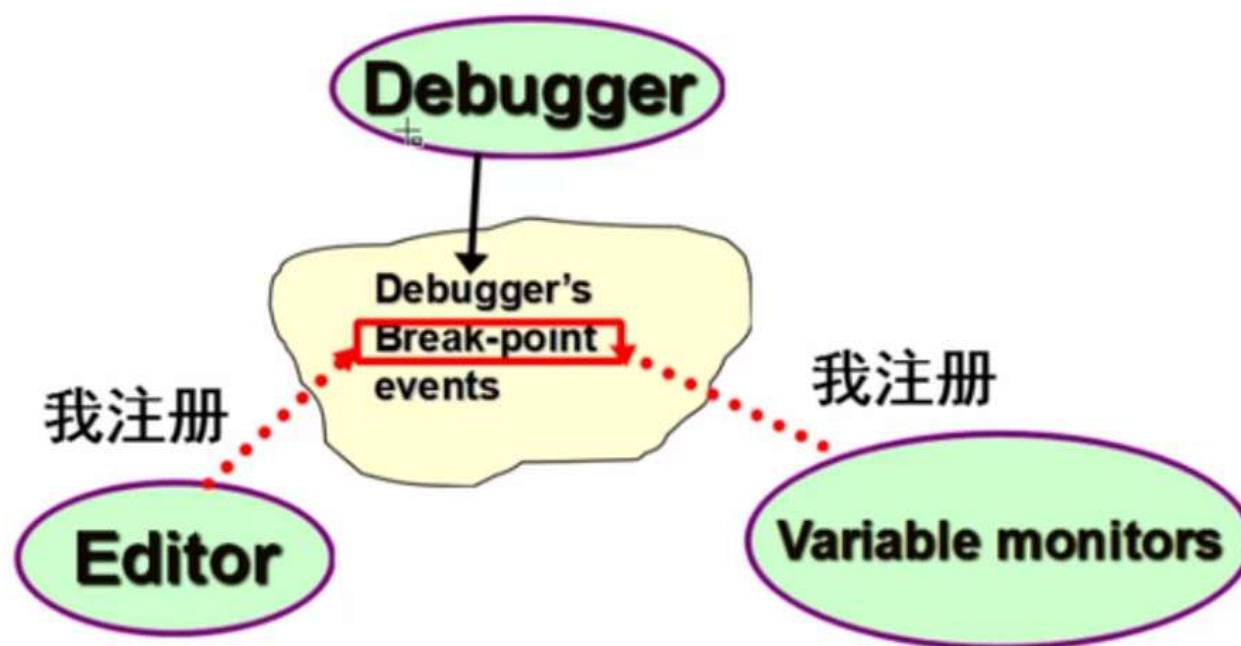
这些IDE 通常包含如下工具

- 代码编辑器Editor,
- 变量监视器Variable Monitor,
- 调试器debugger等.

其实这些IDE即使用了隐式调用架构风格.



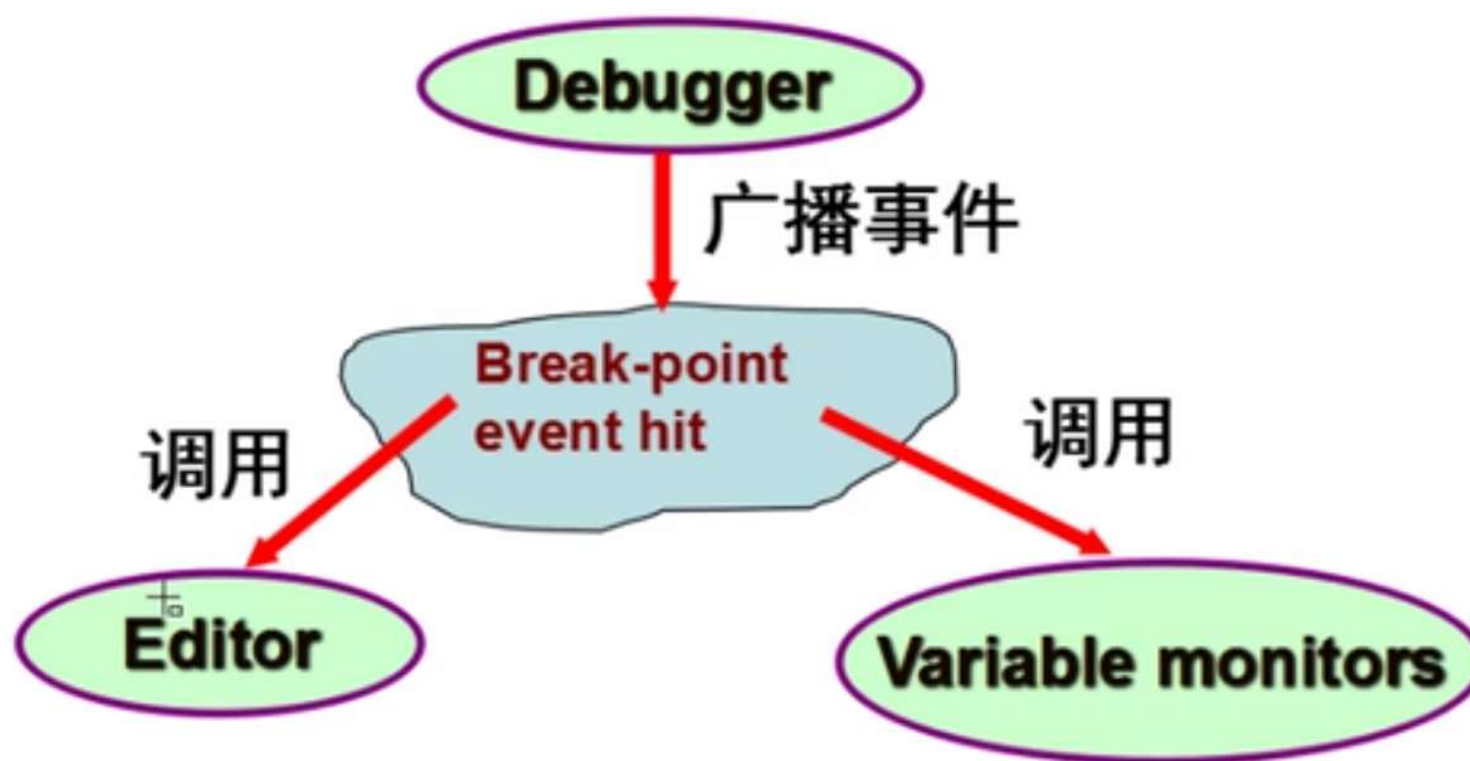
Editors 和 Variable Monitors 注册了 debugger 的 breakpoint 事件.



图例:

.....➡ 注册事件

————➡ 发布事件



事件驱动风格实例

- 应用实例： Win32 GUI程序

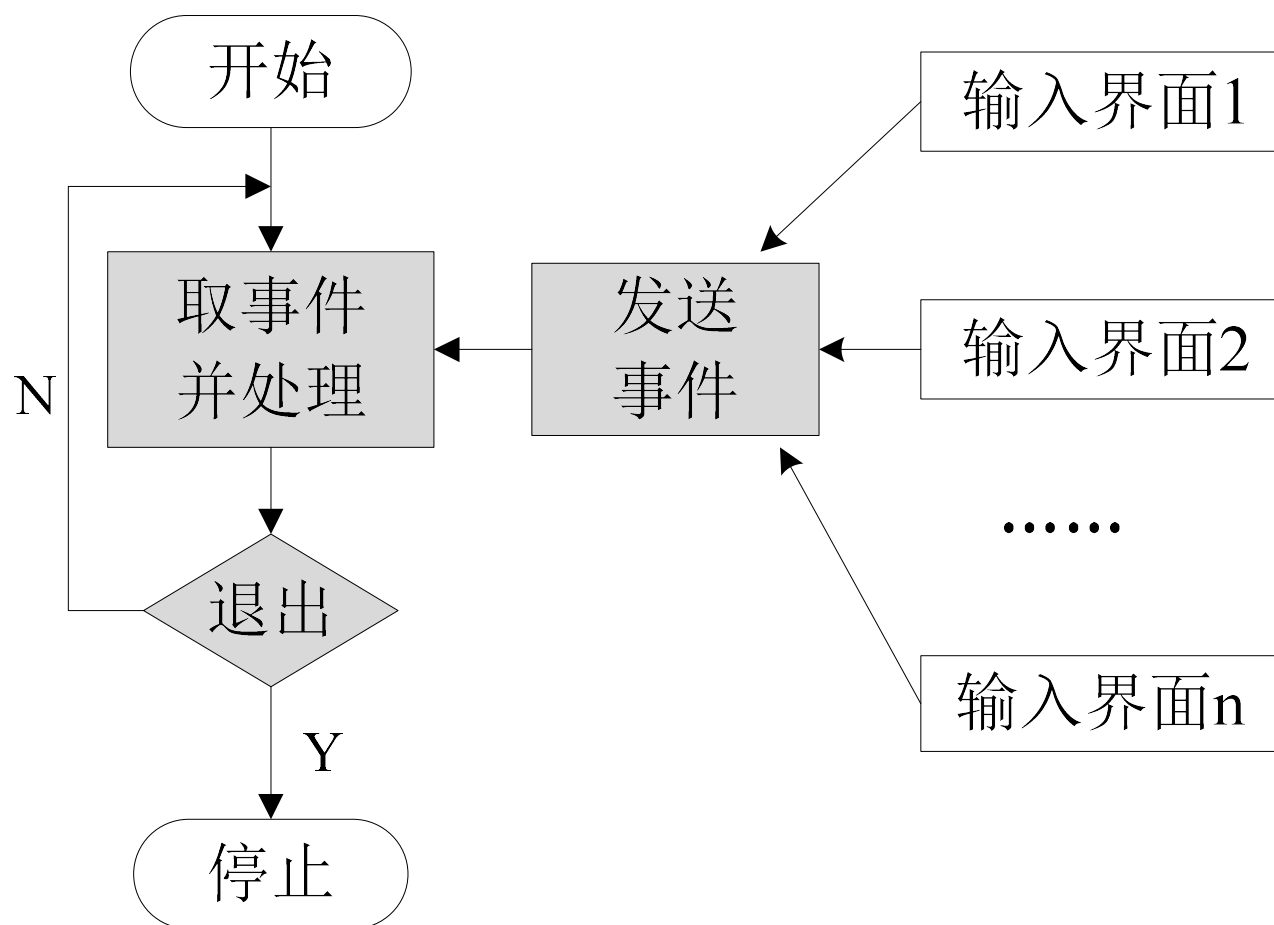


图4-13 Windows事件驱动模型



实例：JavaBean系统

- 事件从事件源到监听者的传递是通过对目标监听者对象的Java方法调用进行的。
- 对每个明确的事件的发生，都相应地定义一个明确的Java方法。
- 这些方法都集中定义在事件监听者（EventListener）接口中，这个接口要继承 `java.util.EventListener`。



实例：JavaBean系统

- 事件状态对象
 - 与事件发生有关的状态信息一般都封装在一个事件状态对象中，这种对象是`java.util.EventObject`的子类。
 - 按设计习惯，这种事件状态对象类的名应以Event结尾。



实例：JavaBean系统

- 事件监听者接口 (EventListener Interface) 与事件监听者
 - 由于Java事件模型是基于方法调用，因而需要一个定义并组织事件操纵方法的方式。JavaBean中，事件操纵方法都被定义在继承了java.util.EventListener类的EventListener接口中，按规定，EventListener接口的命名要以Listener结尾。
 - 任何一个类如果想操纵在EventListener接口中定义的方法都必须以实现这个接口方式进行。这个类也就是事件监听者。



实例：JavaBean系统

- 事件监听者的注册与注销
 - 为了各种可能的事件监听者把自己注册入合适的事件源中，建立源与事件监听者间的事件流，事件源必须为事件监听者提供注册和注销的方法。在前面的bound属性介绍中已看到了这种使用过程，在实际中，事件监听者的注册和注销要使用标准的设计格式：
 - `public void add< ListenerType> (< ListenerType> listener)`
 - `public void remove< ListenerType> (< ListenerType> listener);`



实例：JavaBean系统

- 适配类

- 适配类是JavaBean事件模型中极其重要的一部分。在一些应用场合，事件从源到监听者之间的传递要通过适配类来“转发”。
- 适配类成为了事件监听者，事件源实际是把适配类作为监听者注册入监听者队列中，而真正的事件响应者并未在监听者队列中，事件响应者应做的动作由适配类决定。